



西安电子科技大学
XIDIAN UNIVERSITY



计算机科学与技术学院
SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY
国家示范性软件学院
NATIONAL PILOT SCHOOL OF SOFTWARE ENGINEERING

《软件质量保障大作业》

开题报告

题 目： 基于 PointPCDet 框架的 3D 点云对抗攻击方法复现与优化研究

组 长： 张鸿翔

组 员： 李元庆 邱尹鸿 余瑞洋 高育博

课 程： 《软件质量保障》

《软件质量保障大作业》开题报告组员名单

组员序号	学号	姓名	备注
1	23009201364	张鸿翔	
2	23009200384	李元庆	
3	23009201337	邱尹鸿	
4	23009201052	余瑞洋	
5	23009200242	高育博	
题 目	基于 PointPCDet 框架的 3D 点云对抗攻击方法复现与优化研究		
章 节	一. 研究意义		
	二. 相关现状调研（国内外研究现状）		
	三. 所选择工具初步搭建情况		
	四. 进度安排与预期成效		
	五. 参考文献		

一、研究意义

1. 研究背景与意义

自动驾驶属于典型的安全关键系统，感知模块作为系统“输入端”，其检测结果将直接影响后续决策与控制。一旦出现漏检、误检或检测框偏移等问题，可能导致路径规划和制动策略发生错误，从而带来潜在安全风险。因此，除常规精度指标外，对自动驾驶感知模型开展鲁棒性评估与安全性验证具有重要的工程价值与现实意义。

2. 研究对象与实验平台选择的合理性

本项目聚焦激光雷达（LiDAR）点云 3D 目标检测任务。相比纯视觉方案，点云能够更直接地描述目标的空间几何结构与距离信息，是自动驾驶感知的重要数据来源。我们选择 PointPillars 作为基线检测模型，其具有推理效率高、工程实践成熟等特点，适合进行可复现的对比实验；同时选用 KITTI 作为评测数据集，该数据集是自动驾驶 3D 检测领域的经典公开基准，便于在统一标准下进行复现与结果对齐。为保证实现的工程可落地性和实验流程的规范性，本项目基于 OpenPCDet 框架完成模型推理、攻击注入与结果可视化，从而搭建稳定的实验底座。

3. 理论依据与任务拓展价值（从分类攻击到检测验证）

项目主要参考 CVPR 2019 论文《Generating 3D Adversarial Point Clouds》（Chong Xiang, Charles R. Qi, Bo Li）提出的点云对抗攻击思路，包括点坐标微扰、点删除与点添加/生成等典型方式，并结合几何约束（如 Chamfer、Hausdorff 距离）实现“扰动幅度受控”的攻击样本生成。需要指出的是，相关经典研究多在 PointNet/PointNet++ 等分类或分割模型上验证，而自动驾驶的核心任务是 3D 目标检测，其输出不仅包含类别，还涉及三维框定位与置信度，失效形式更复杂。将对抗攻击方法迁移到 PointPillars + KITTI 的检测场景并进行系统化验证，能够更贴近真实应用需求，为理解检测模型在输入扰动下的脆弱性提供更直接的证据。

4. 面向软件质量保障的课程价值

本项目的意义不仅在于实现攻击算法，更在于建立一套可复现的验证流程：在 OpenPCDet 的推理数据流中完成攻击注入，结合 BEV 可视化与定量对比，观察攻击前后漏检/误检及定位偏差等变化。该流程有助于把 AI 系统测试从“只看准确率”拓展到“关注可靠性与安全性”，并为后续轻量级防御（如点云预处理过滤等）提供实验基础与评估依据。

二、 相关现状调研（国内外研究现状）

本课题聚焦于自动驾驶 3D 点云感知算法的安全性，目前学术界与工业界在该领域的研究现状主要分为以下四个维度：

1. 3D 目标检测模型与工业界应用现状（靶机调研）

近年来，基于深度学习的 3D 目标检测算法取得了突破性进展。早期的 PointNet 系列开创了直接处理无序点云的先河，但由于计算复杂度高，难以直接应用于包含十余万个点的真实自动驾驶大场景。随后，工业界逐渐演进出基于体素（Voxel）和柱体（Pillar）的高效体素化网络，其中以 PointPillars 为代表的模型因其极高的推理速度和检测精度，成为了目前自动驾驶（如蔚来、小鹏等企业）最主流的基线算法[1]。

在最新进展方面，KITTI 排行榜（截至 2025 年）显示，当前最先进的 3D 检测方法在 Car 类 Moderate 难度上已达到 86-87% 的平均精度，较早期 PointPillars 的约 79% 有显著提升。同时，多模态融合（激光雷达+相机）正成为新的技术趋势[2]。

在工程落地方面，目前以 OpenPCDet 为代表的开源纯 PyTorch 检测框架整合了绝大多数主流网络，为算法的统一评估和安全性测试提供了成熟的底层工业引擎[2]。

2. 三维点云对抗攻击技术现状（攻击算法调研）

随着 2D 图像对抗攻击（如 FGSM、PGD）的成熟，研究者的视线逐渐转移到 3D 物理空间的安全性上。与 2D 规则像素矩阵不同，3D 点云具有稀疏性和无序性，这为对抗攻击带来了新的挑战[3]。

该领域的奠基性工作主要以 CVPR 2019 的经典文献《Generating 3D Adversarial Point Clouds》为代表。该研究系统性地将点云攻击划分为三大类：

点坐标微扰（Perturbation）：通过计算模型梯度，使点云发生肉眼不可见的微小空间位移。

点生成/插入（Generation/Insertion）：在空旷区域合成独立的对抗点簇，引发误检。

关键点删除（Dropping）：利用显著性图评估点的重要性，精准移除特征

点导致漏检[9]。

近两年的研究呈现出以下几个重要趋势：

从数字域到物理世界的攻击迁移。 近期研究已不再满足于在数字空间中生成对抗样本，而是积极探索攻击在真实物理世界中的可实现性。2025 年 USENIX Security 会议上发表的 "Shadow Hack" 工作首次利用激光雷达难以精确测量的材料（如镜面反射片、红外截止滤光片），在点云中制造"对抗阴影"，在 11-21 米距离上对 PointPillars 和 SECOND-IoU 等模型实现了接近 100% 的攻击成功率，并在真实物理环境中验证了可行性[4]。这类攻击的隐蔽性极强，因为攻击材料本身在点云中几乎不可见，却能通过改变周围环境的激光反射模式引发误检。

对抗训练与鲁棒性增强。 针对对抗攻击的防御，研究者也从模型侧提出了增强方案。2025 年提出的 CGAL（Curriculum-Guided Adversarial Learning）框架，通过课程学习引导的对抗训练策略，结合基于径向基函数网络（RBFN）的非线性增强模块（Pillar-RBFN），使 PointPillars 在不进行对抗训练的情况下即具备内在的对抗鲁棒性。该方法还构建了 Adv-KITTI 对抗数据集，在 KITTI 上实现了 0.8~2.5 个百分点的 mAP 提升[5]。

多目标定向攻击。 早期攻击多为非定向（让模型任意误分类），而近期研究已扩展到定向攻击和多目标攻击。例如，有研究提出了"双目标对抗样本"生成方法，能使不同模型分别被诱导至攻击者指定的不同类别[6]。

通用对抗扰动（UAP）。 与针对单个样本的攻击不同，通用对抗扰动是一种与输入无关的扰动，可对任意输入样本生效，对实际部署的自动驾驶系统构成更大威胁。最新的防御研究已开始关注 UAP 的结构性特征，利用图小波变换等方法识别并移除对抗区域[7]。

可微渲染驱动的新型攻击范式。 随着 NeRF（神经辐射场）和 3D 高斯泼溅（3D Gaussian Splatting）等可微渲染技术的兴起，攻击者现在可以优化 3D 场景参数（如物体纹理、几何形状、光照条件）来生成物理上可实现的对抗样本，这开辟了从"点云扰动"到"3D 场景操纵"的全新攻击维度[8]。

3. 鲁棒性评估与防御策略现状（防御算法调研）

面对日益严峻的对抗攻击威胁，研究者从多个层面提出了防御策略：

数据预处理层防御。目前最轻量级的防御思路是在点云输入检测网络之前，采用统计学滤波（如统计离群点移除 SOR、半径离群点移除 ROR）来过滤掉由攻击算法生成的异常噪声点[7]。这种方法能够在不修改底层检测网络权重的前提下，以较低的算力成本抵御一定程度的攻击扰动。但最新研究表明，这类方法对"结构化"的通用对抗扰动（UAP）效果有限，因为 UAP 并非孤立的噪声点，而是具有连续曲率的连贯几何结构[7]。

对抗训练。如前述 CGAL 框架所示，通过在训练数据中注入对抗样本，可显著提升模型的鲁棒性[5]。但对抗训练的代价是训练时间大幅增加，且对未见过的攻击类型泛化能力有限。

基于模型可解释性的防御。最新的防御范式开始利用模型可解释性技术，识别对抗样本中对分类器预测影响最大的区域，并通过"移植测试"（将可疑区域移植到干净样本上观察是否引发误分类）来区分真实几何特征与对抗结构，再进行定向移除[7]。这种"模型引导+结构感知"的防御策略代表了当前最先进的防御方向。

物理层防御。针对物理世界攻击（如 Shadow Hack），研究者提出了 BBValidator 等防御机制，通过验证检测框的合理性（如检查是否存在与遮挡模式一致的阴影），实现了 100% 的攻击检测率，同时保持了较高的目标检测精度[4]。

4. 现状总结与本课题的切入点

综上所述，当前 3D 点云对抗攻击领域呈现以下态势：

攻击理论日趋成熟：从早期的数字域坐标微扰[3]，发展到物理世界可实现的阴影攻击[4]、可微渲染驱动的场景操纵[8]等新范式，并涌现出以点云微扰与点删除为代表的经典攻击方法[9]。

防御手段逐步多元化：从单一的预处理滤波，扩展到对抗训练[5]、模型可解释性引导的结构化防御[7]等多个层面。

评估基准更加系统化：KITTI 数据集持续作为标准评测平台，OpenPCDet 框架提供了统一的模型训练与评估环境[2]。

但目前仍存在以下痛点：

工程落地壁垒：大部分攻击研究基于独立的分类网络（如 PointNet）或简

化的实验环境，将攻击方法迁移到工业级检测框架（如 OpenPCDet）中需要解决数据预处理流程差异、多坐标系转换、梯度回传接口不统一等工程问题。

系统性评估缺失：现有评估多关注分类准确率下降，而在自动驾驶核心任务——3D 目标检测中，针对漏检率（Miss Rate）、误检率（False Positive Rate）及定位误差（Localization Error）的系统性安全评估仍不充分。

攻防对抗的完整性不足：多数研究要么只做攻击，要么只做防御，缺乏在同一框架下完成"攻击→评估→防御→再评估"的完整闭环实验。

本课题正是切入上述痛点，旨在：基于 OpenPCDet 工业级框架，在 KITTI 真实街景数据集上，复现并改进经典攻击算法（如坐标微扰、点生成/删除），对 PointPillars 等主流检测模型完成系统化的漏检/误检安全性评估，并探索 SOR/ROR 滤波等轻量级防御策略的有效性，打通"理论攻击算法→工业检测框架→安全性验证报告"的完整工程链路。

三、所选择工具初步搭建情况

一、实验环境配置

1.1 硬件环境

显卡

```
root@autodl-container-47fd48955e-2f55cea8:~# nvidia-smi
Thu Apr  2 23:10:58 2026
+-----+
+---+
| NVIDIA-SMI 580.76.05                Driver Version: 580.76.05   CUDA Version: 13.0
+---+
+-----+
+---+
| GPU   Name                               Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. E
CC | Fan  Temp  Perf    Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute
M. |                                         |                  |                     |
M. |                                         |                  |                     |
+---+-----+
===|
|  0  NVIDIA GeForce RTX 4090             On | 00000000:B8:00:0  Off |
ff |                                         | 0MiB / 24564MiB | 0%      Defau
lt |                                         |                  |
|                                         |                  |
/A |                                         |                  |
+---+-----+
```

图 3-1

1.2 软件环境

实验环境

实验需要 cuda11.8, python3.8, ubuntu20.04。

```
root@autodl-container-47fd48955e-2f55cea8:~# python --version
Python 3.8.10
root@autodl-container-47fd48955e-2f55cea8:~# nvcc --version
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2022 NVIDIA Corporation
Built on Wed_Sep_21_10:33:58_PDT_2022
Cuda compilation tools, release 11.8, V11.8.89
Build cuda_11.8.r11.8/compiler.31833905_0
root@autodl-container-47fd48955e-2f55cea8:~# conda env list
# conda environments:
#
base * /root/miniconda3
```

图 3-2

```

root@autodl-container-47fd48955e-2f55cea8:~# cat /etc/os-release
NAME="Ubuntu"
VERSION="20.04.5 LTS (Focal Fossa)"
ID=ubuntu
ID_LIKE=debian
PRETTY_NAME="Ubuntu 20.04.5 LTS"
VERSION_ID="20.04"

```

图 3-3

1.3 虚拟环境

由于使用 autodl 平台，自选了带有了实验环境数据集的镜像，环境选择 base，不创建额外虚拟环境。

1.4 核心依赖

验证 PyTorch 和 CUDA，spconv。

```

root@autodl-container-47fd48955e-2f55cea8:~# # 验证PyTorch和CUDA
root@autodl-container-47fd48955e-2f55cea8:~# python -c "import torch; print('PyTorch:'
torch.__version__); print('CUDA可用:', torch.cuda.is_available())"
PyTorch: 2.0.0+cu118
CUDA可用: True
root@autodl-container-47fd48955e-2f55cea8:~# # 验证spconv
root@autodl-container-47fd48955e-2f55cea8:~# python -c "import spconv; print('spconv版
本:', spconv.__version__)"
spconv版本: 2.3.6

```

图 3-4

二、数据集准备

2.1 KITTI 数据集介绍

KITTI 数据集是自动驾驶领域最常用的 3D 目标检测基准数据集，包含：

- ✧ 7481 个训练样本
- ✧ 7518 个测试样本
- ✧ 标注类别：Car、Pedestrian、Cyclist

2.2 数据集解压

使用的含有数据集压缩包的镜像，数据集很大，有 41GB

创建数据目录

```
mkdir -p /root/autodl-tmp/kitti
```

解压相机标定文件

```
unzip -q /root/autodl-pub/KITTI/object/data_object_calib.zip -d /root/au
todl-tmp/kitti/
```

解压标注文件

```
unzip -q /root/autodl-pub/KITTI/object/data_object_label_2.zip -d /root/autodl-tmp/kitti/
```

解压点云数据（约 28.7GB）

```
unzip -q /root/autodl-pub/KITTI/object/data_object_velodyne.zip -d /root/autodl-tmp/kitti/
```

```
root@autodl-container-47fd48955e-2f55cea8:~# ls -la /root/autodl-tmp/kitti/
total 0
drwxr-xr-x 4 root root 49 Apr  2 22:34 .
drwxr-xr-x 4 root root 46 Apr  2 22:33 ..
drwxr-xr-x 4 root root 47 Apr  2 22:37 testing
drwxr-xr-x 5 root root 66 Apr  2 22:34 training
root@autodl-container-47fd48955e-2f55cea8:~# ls /root/autodl-tmp/kitti/training/velodyne/ | wc -l
7481
root@autodl-container-47fd48955e-2f55cea8:~#
```

图 3-5

三、OpenPCDet 框架安装

3.1 克隆代码仓库

```
cd /root
git clone https://github.com/open-mmlab/OpenPCDet.git
cd OpenPCDet
```

创建数据软链接

```
ln -s /root/autodl-tmp/kitti data/kitti
```

创建软连接，解决磁盘空间问题，数据存在大盘（autodl-tmp），代码从小盘（OpenPCDet）访问数据，不需要复制 28GB 数据，节省空间和时间

```
root@autodl-container-47fd48955e-2f55cea8:~# cd /root/OpenPCDet
root@autodl-container-47fd48955e-2f55cea8:~/OpenPCDet# ls -la data/
total 4
drwxr-xr-x  6 root root   93 Mar 22 23:17 .
drwxr-xr-x 12 root root 4096 Mar 22 17:37 ..
drwxr-xr-x  3 root root   31 Mar 22 16:51 argo2
lrwxrwxrwx  1 root root    22 Mar 22 23:17 kitti -> /root/autodl-tmp/kitti
drwxr-xr-x  3 root root   31 Mar 22 16:51 lyft
drwxr-xr-x  3 root root   31 Mar 22 16:51 once
drwxr-xr-x  3 root root   31 Mar 22 16:51 waymo
root@autodl-container-47fd48955e-2f55cea8:~/OpenPCDet#
```

图 3-6

3.2 安装 OpenPCDet

```
cd /root/OpenPCDet
pip install -r requirements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple
```

python setup.py develop

```
root@autodl-container-47fd48955e-2f55cea8:~/OpenPCDet# cd /root/OpenPCDet
root@autodl-container-47fd48955e-2f55cea8:~/OpenPCDet# python -c "import pcdet; print('
OpenPCDet版本:', pcdet.__version__); print('安装路径:', pcdet.__file__)"
OpenPCDet版本: 0.6.0+233f849+py7327674
安装路径: /root/OpenPCDet/pcdet/__init__.py
root@autodl-container-47fd48955e-2f55cea8:~/OpenPCDet#
```

图 3-7

四、模型权重准备

4.1 下载 PointPillar 预训练权重

PointPillar 是一种高效的点云检测模型，将点云转换为伪图像进行处理

```
root@autodl-container-47fd48955e-2f55cea8:~/OpenPCDet# ls -lh /root/OpenPCDet/ckpts/
total 19M
-rw-r--r-- 1 root root 19M Mar 22 18:07 pointpillar_7728.pth
root@autodl-container-47fd48955e-2f55cea8:~/OpenPCDet#
```

图 3-8

五、运行 Demo 测试

5.1 执行目标检测

cd /root/OpenPCDet/tools

conda activate pcdet

python demo.py |

--cfg_file cfgs/kitti_models/pointpillar.yaml |

--ckpt ../ckpts/pointpillar_7728.pth |

--data_path ../data/kitti/testing/velodyne/000008.bin

```
1.0702100, -1.3302100]], device='cuda:0')
置信度得分:
tensor([0.9728, 0.9425, 0.9251, 0.9161, 0.8112, 0.7411, 0.6463, 0.6096, 0.5677,
        0.5071, 0.4973, 0.4696, 0.4508, 0.4447, 0.4198, 0.3726, 0.3720, 0.3507,
        0.3333, 0.3316, 0.3314, 0.3290, 0.3093, 0.2825, 0.2816, 0.2537, 0.2532,
        0.2455, 0.2329, 0.2079, 0.2020, 0.1983, 0.1922, 0.1790, 0.1768, 0.1752,
        0.1729, 0.1596, 0.1584, 0.1558, 0.1501, 0.1496, 0.1425, 0.1340, 0.1326,
        0.1265, 0.1236, 0.1168, 0.1157, 0.1144, 0.1104, 0.1094, 0.1072, 0.1065,
        0.1053, 0.1046, 0.1020, 0.1015], device='cuda:0')
类别标签 (1=Car, 2=Pedestrian, 3=Cyclist):
tensor([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 1, 1, 1, 2, 1, 3, 1, 2, 1, 1,
        3, 2, 3, 1, 1, 1, 1, 2, 1, 2, 2, 1, 3, 2, 1, 3, 2, 2, 1, 3, 2, 1, 2,
        3, 2, 2, 3, 2, 2, 1, 2, 2, 1], device='cuda:0')

🐦 鸟瞰图已成功渲染并保存至: /root/OpenPCDet/tools/result_bev.png
2026-04-02 23:29:53,909 INFO Demo done.
root@autodl-container-47fd48955e-2f55cea8:~/OpenPCDet/tools# |
```

图 3-9

运行结果图

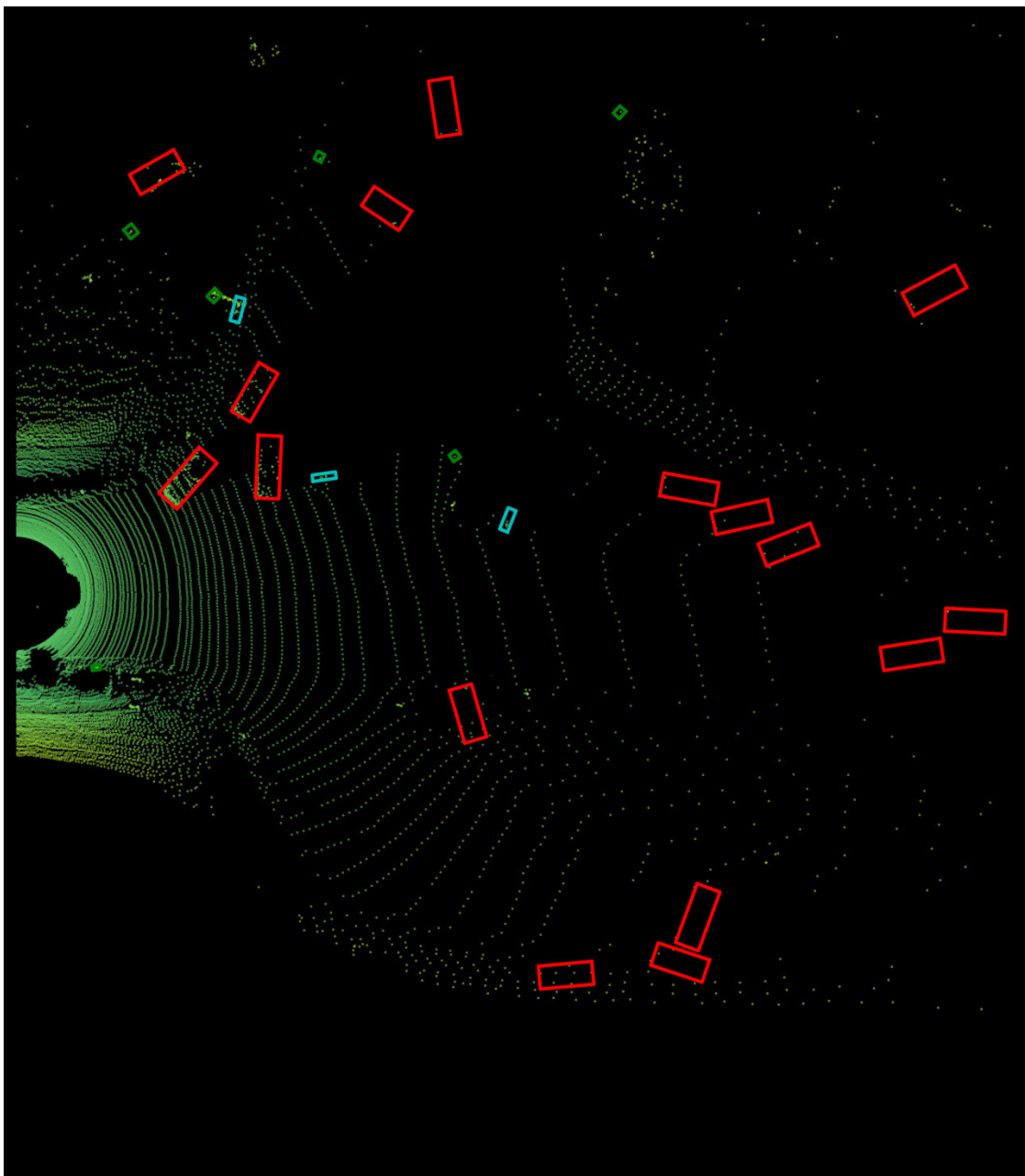


图 3-10

- ✓ 红色检测框准确标注了车辆位置
- ✓ 绿色检测框识别出行人
- ✓ 蓝色检测框标注了骑行者
- ✓ 背景点云以渐变色显示，颜色越亮表示高度越高
- ✓ 最高检测置信度达到 97.28%

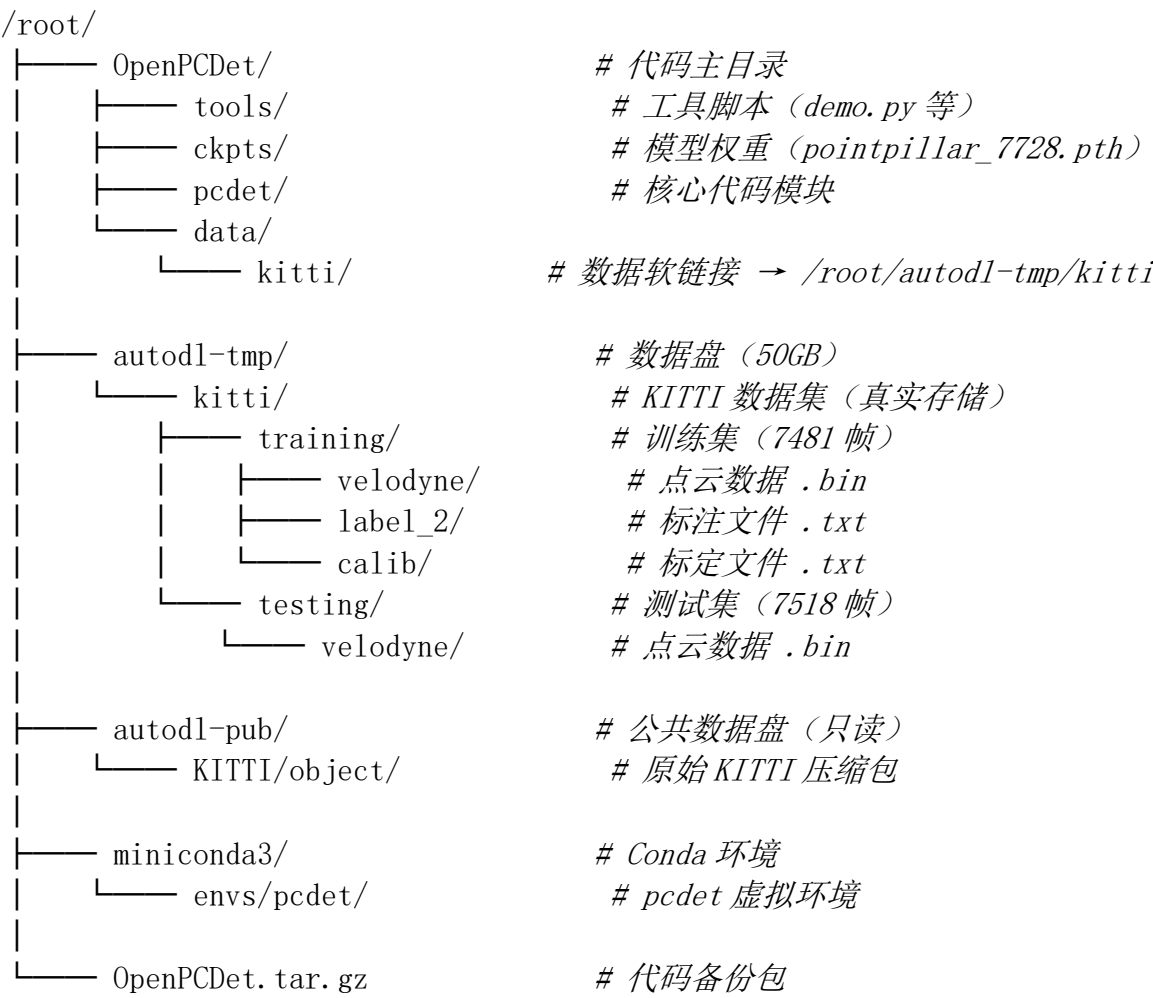
六、环境配置总结

6.1 配置清单

表 3-1

组件	版本/规格
GPU	RTX 4090 24GB
CUDA	11.8
Python	3.8.10
Ubuntu	20.04
PyTorch	2.0.1
spconv	2.3.6
数据集	KITTI (7481 训练 / 7518 测试)
模型	PointPillar

6.2 关键路径



```

/root/
├─ OpenPCDet/
│   ├─ tools/
│   ├─ ckpts/
│   ├─ pcdet/
│   └─ data/
│       └─ kitti/
│
├─ autodl-tmp/
│   └─ kitti/
│       ├─ training/
│       │   ├─ velodyne/
│       │   └─ label_2/
│       │       └─ calib/
│       └─ testing/
│           └─ velodyne/
│
├─ autodl-pub/
│   └─ KITTI/object/
│
├─ miniconda3/
│   └─ envs/pcdet/
│
└─ OpenPCDet.tar.gz

```

代码主目录
 # 工具脚本 (demo.py等)
 # 模型权重 (pointpillar_7728.pth)
 # 核心代码模块

 # 数据软链接 → /root/autodl-tmp/kitti

 # 数据盘 (50GB)
 # KITTI数据集 (真实存储)
 # 训练集 (7481帧)
 # 点云数据 .bin
 # 标注文件 .txt
 # 标定文件 .txt
 # 测试集 (7518帧)
 # 点云数据 .bin

 # 公共数据盘 (只读)
 # 原始KITTI压缩包

 # Conda环境
 # pcdet虚拟环境

 # 代码备份包

图 3-11

四、进度安排与预期成效

一、进度安排

本次课程作业分为以下四个阶段，具体时间安排见表 4-1。以下是各个阶段的具体安排。

阶段一：搭建宿主环境与确立基准（已完成）

1. 框架部署：在 AutoDL 云服务器（vGPU）上克隆并编译 OpenPCDet 框架，配置 CUDA 11.8 + PyTorch 2.0.0 + Python 3.8。数据挂载：解压并挂载 KITTI 3D 数据集（41GB，含 7481 训练样本/7518 测试样本），通过软链接解决磁盘空间问题。

2. 基线验证：运行 PointPillars 预训练权重，在原始点云上输出 3D 检测框。

阶段二：实施“拦截器”攻击策略

1. 确立拦截点：在 tools/demo.py 中追踪数据流，定位到 data_dict['points']（点云张量喂给模型前的那一刻）

2. 提取攻击逻辑：从 Robust3DOD 仓库中提取以下三类攻击的核心 PyTorch 张量操作代码：

点坐标微扰（Perturbation）：沿梯度方向对现有点添加微小位移

点插入（Generation）：在目标附近生成对抗点簇

关键点删除（Dropping）：按显著性图移除特征点

3. 代码注入：将攻击函数作为补丁粘贴至 demo.py，在 model.forward() 之前调用，生成“对抗性数据字典”。

阶段三：攻击效果评估与可视化

1. BEV 对比可视化：修改渲染器，生成攻击前后的鸟瞰图对

（result_bev_clean.png 和 result_bev_attacked.png）

定量评测：漏检率/误检率/置信度下降等指标

2. 参数化实验：在不同扰动强度（ ϵ ）、不同攻击类型下重复实验，生成敏感性分析曲线

阶段四：轻量级防御探索与报告撰写（实验改进点）

1. 防御验证：在数据预处理层引入 SOR（Statistical Outlier Removal）或 ROR（Radius Outlier Removal）滤波

- 2.攻防闭环：对比“无防御 + 攻击”与“有防御 + 攻击”的评测指标，验证轻量级防御的有效性
- 3.文档撰写：撰写结题报告，整理代码仓库。

表 4-1

阶段	时间	主要任务	对应实验内容
阶段一	第 1-5 周	搭建宿主环境与开题报告	OpenPCDet 框架部署 + KITTI 数据集挂载 + PointPillars 基线验证
阶段二	第 6-7 周	实施“拦截器”攻击策略	定位数据拦截点 + 提取/改写攻击逻辑 + 代码注入
阶段三	第 8-10 周	攻击效果评估与可视化	BEV 对比可视化 + 定量评测（漏检率/误检率/置信度下降）
阶段四	第 11-14 周	轻量级防御探索与报告撰写	SOR/ROR 滤波防御验证 + 攻防闭环分析 + 结题报告

二、预期成效

2.1 论文复现

成功复现经典攻击：在 OpenPCDet 工业级框架上复现 CVPR 2019 《Generating 3D Adversarial Point Clouds》的三类点云对抗攻击方法（点坐标微扰、点插入、关键点删除）

任务迁移验证：将原论文在“3D 形状分类”任务上的攻击方法，迁移至“自动驾驶 3D 目标检测”任务

检测模型失效分析：总结检测模型在对抗攻击下的失效表现（漏检、误检、框偏移、置信度崩溃等）

使用 BEV 可视化工具：攻击前后对比的可视化渲染器，直观展示检测框变化

2.2 安全评估层面

定量评测报告：生成点云对抗攻击对 PointPillars 模型的安全性评估报告，包含：不同攻击类型下的漏检率/误检率变化曲线；扰动强度与检测精度（mAP）的敏感性分析；不同目标类别（Car / Pedestrian / Cyclist）的脆弱性对比。

轻量级防御验证：评估 SOR/ROR 滤波在不同攻击强度下的防御效果，分析其适用边界与局限性。

五、参考文献

- [1] LANG A H, VORA S, CAESAR H, et al. PointPillars: fast encoders for object detection from point clouds[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. Long Beach: IEEE, 2019: 12697-12705.
- [2] GEIGER A, LENZ P, STILLER C, et al. Vision meets robotics: the KITTI dataset[J]. International Journal of Robotics Research, 2013, 32(11): 1231-1237.
- [3] NADERI H, BAJIĆ I V. Adversarial attacks and defenses on 3D point cloud classification: a survey[J]. IEEE Access, 2023, 11: 144274-144295.
- [4] KOBAYASHI R, NOMOTO K, TANAKA Y, et al. Invisible but detected: physical adversarial shadow attack and defense on LiDAR object detection[C]//Proceedings of the 34th USENIX Security Symposium. Seattle: USENIX, 2025.
- [5] HUANG J, XIE Y, CHEN Z, et al. Curriculum-guided adversarial learning for enhanced robustness in 3D object detection[J]. Sensors, 2025, 25(6): 1697.
- [6] LEE T, LEE S, KWON H. Dual-targeted adversarial noise for 3D point cloud classification model[J]. Scientific Reports, 2025.
- [7] GAO Y, CHANG X, LI H, et al. Segments-aware universal adversarial perturbations purification on 3D point cloud classifiers[J]. Frontiers in Computer Science, 2025, 7: 1626359.

[8] HULL M, WANG H, LAU M, et al. RenderBender: a survey on adversarial attacks using differentiable rendering[C]//Proceedings of the 33rd International Joint Conference on Artificial Intelligence. 2025: 10473-10480.

[9] HE F, CHEN Y, CHEN R, et al. Point cloud adversarial perturbation generation for adversarial attacks[J]. IEEE Access, 2023, 11: 2767-2774.